



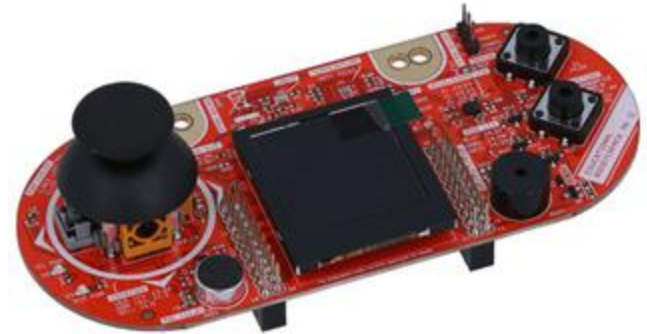
Embedded Software for the Internet Of Things

Final Lecture

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Topics Covered

- Embedded Systems Fundamentals
 - ARM Cortex Architecture, Microcontrollers, Peripherals.
- Practical Coding (MSP432)
 - Interrupts, Sensors, Timers..
- Embedded C Programming
 - Compilers, Optimization...
- Real-time Concepts
 - Scheduling, FreeRTOS



Topics Covered

- We covered some topics **in detail**
 - MCUs, peripherals,...
- Some others **at a high level**
 - Compilers and optimizations, RTOS, ...
- We had **intersections** with many other disciplines:
 - Computer arch, programming languages, operating systems, many other topics

Topics Covered

- Now, you are familiar with (at least):
 - the 'existence' of the topics covered in the course:
- You have a **concrete map** in your mind!
 - You can return back to these topics, check other resources, deepen your knowledge...
- You can take more sophisticated courses and spend years to be **an expert!**
- You can program any microcontroller!

High-level vs. low-level programming

High-level

- **Advantages:**

- Higher productivity (less development time)
- Portability across devices
- Code is easier to read and maintain
- Allows code reuse
- Rapid prototyping of applications

- **Disadvantages:**

- Less optimized code
- Additional translation time from source to machine code
- Another level of abstraction to deal with

Low-level

- **Advantages:**

- More optimized code and higher memory efficiency
- Less translation time from source to machine code
- Direct interaction with hardware

- **Disadvantages:**

- Less portability from one device to another
- Code is more difficult to read, reuse, and maintain
- Low productivity

You...

- Are **not a newbie** now!
- Already passed the '**hobbyist level**' and became **a real engineer!**
- Can develop **any IoT** and **any embedded systems applications** now.
- Have the fundamentals for embedded systems **research** and **industry**.

Course grading

%30 Written Exam + **%70** Project Evaluation



Written Exam

- **15** multiple-choice questions to be completed within 45 minutes.
 - The value of each question is **1.00** points in the case of the correct answer,
 - **-0.20** points in the case of the wrong answer, and
 - 0 points in the case of no answer (to be explicitly selected).
- The maximum grade is **15.00** points.
- The **minimum** grade to pass the exam is **7.0** points.

Written Exams and Project Control

- **EXAM-1:** 28/01/2026 10:30 Aula pc B106
- **EXAM-2:** 18/02/2026 10:30 Aula pc A202
 - The *tentative* project presentation dates for this session will be:
 - 26/01/2026
 - 27/01/2026

Register yourself on ESSE3

Exams And Projects

- If you are registered for Exam-1, took the exam but you could not present your project:
 - I will **keep** your exam vote internally.
 - Then, **you have to register** for the **Exam-2** to present your project.
 - Then I will register the final vote for that exam session.
- **In Summary:**
 - If you register for an exam session but do not present your project before that exam, you need to register for another exam session to present your project and get your vote.

Project Groups

- Project group: composed of mostly 3-4 students
- Pair programming as much as possible.
- All members should:
 - put equal effort!
 - have **full control** over the project.
- **Every member of the group is responsible for each part of the system!**



Project Evaluation - Max 15 minutes

- **Maximum 2-minute pitch video** demonstration
- **Maximum 5-minute presentation**
 - Prepare **maximum 5-6 slides**
 - **All** team members will present -> Split presentation into equal parts.
- Approximately 5 minutes Q&A.
- I will ask technical questions to all team members **separately**
 - Where? -> In my room (**Office 120**, Povo 2)

2-Minute Pitch Video

- **Problem Statement**

- *Describe which problem your system solves.*

- **Basic Working Scheme**

- *Describe how your system works.*
 - *Which sensors did you use? Explain the basic working flow of the system.*
 - *If applicable, explain how does the user interacts with your system.*

5-MIN. PRESENTATION

- **Software Architecture**

- Describe the **core software blocks** of your system.
 - Explain the working flow of the software by giving details on the **hardware/software interaction**:
 - e.g., which **interrupts** you used, how you serviced the interrupts?
Explain how you sense data, how do you interpret and process the data.
 - Indicate what kind of **data structures** and **algorithms** you used within your code.

5-MIN. PRESENTATION

- Team Member contributions:
 - I prefer everything to be developed **ALLTOGETHER**.
 - If development is **separated** among team members:
 - Write who did which part of the project.
 - Present how many lines of code each member contributed.
 - **EVERYONE is responsible from EVERY SINGLE LINE.**
- Representative Codes:
 - Present some core C codes for your implementation - > the most important and representative lines (just a couple of lines)

5-MIN. PRESENTATION

- **Testing**
 - *Describe how did you test the software.*
 - *What kind of problems did you encounter? How did you solve them?*
- **Conclusions and Future Work**
 - *How would you improve your system?*
 - *E.g., what additional features and which additional sensors would you add?*

PROJECT GRADING

- **Project Evaluation**

- Project demonstration and presentation **%60** -> **all team members**
 - Presentation quality **%10**
 - **Pay attention to address all the sections in the presentation.**
 - Github repository quality **%10**
 - Quality of the project **%40**
 - How sophisticated the project is (software-wise, algorithms...)
- Technical questions and answers. **%40** -> **individual points**

What to deliver?

- Create a **public** Github Repository
- Your repository should include:
 - **Source code** of your project
 - Including the source code of any other external code
 - **Powerpoint presentation** (or any other format that can be modified)
 - A **Readme** file explaining your project:
 - Check the next slide!

What to deliver? - **Readme file**

- Your **Readme** file should include the following:
 - What are needed to run the project:
 - Hardware/software requirements
 - Project Layout
 - Source code organization
 - How to build, burn and run your project
 - User Guide
 - Links to your powerpoint presentation and to your Youtube video
 - Team members:
 - explaining clearly how the members contributed to the project
- An example Readme: check [here](#) or othe sample projects.

Sample Questions for the Written Exam

Questions...

- Which of the following statements is **correct** regarding the characteristics of embedded systems?
 - Failure to meet a deadline is unsafe and can even endanger lives.
 - Size, power consumption, and cost of building the system is always more important than its functionality.
 - The ability of the system to meet deadlines is always more important than its energy consumption.
 - The requirements of embedded systems are exactly the same as the needs of general-purpose computers.
 - None of the given options

Questions...

- Which of the following statements is **incorrect** regarding the software development for embedded systems?
 - Software for an embedded system is developed on one platform but runs on another.
 - A host system is the system on which the embedded software is developed.
 - A target system is the embedded system under development.
 - A cross compiler runs on one type of processor architecture but produces object code for a different type of processor architecture
 - None of the given options

Questions

- Which of the following is not an advantage of **high-level** programming?
 - Portability across devices
 - Code is easier to read and maintain
 - Allows code reuse
 - Rapid prototyping of applications
 - Highly-optimized code

Questions

- Which of the following is not a property of **low-level** programming?
 - More optimized code and higher memory efficiency
 - Less translation time from source to machine code
 - Direct interaction with hardware
 - Less portability from one device to another
 - none of the above

Questions...

- Which of the below is **incorrect** for a Harvard style machine?
 - The program counter (PC) points to data memory
 - Does not support self-modifying code
 - Allows two simultaneous memory fetches since it has two memories with separate ports
 - Used by most DSPs since the separation of program and data memories provides higher performance
 - None of the given options

Questions...

- Which option below is **incorrect**?
 - There can be only one microcontroller on an embedded platform
 - Major components of a generic computing platform are the CPU, bus, memory, and I/O devices
 - Component cost, physical size, weight, heat dissipation, support for required I/O devices or memory are the hardware factors that might be considered when choosing a computing platform
 - A microcontroller is one form of a single-chip computer that includes a processor, memory, and I/O devices
 - None of the given options

Questions...

- Assume that the code below is the timer interrupt handler routine. Select the **incorrect** option from below:

```
void interrupt timer_handler() {  
    if (T0IE && T0IF) { /* timer 0 interrupt */  
        blink(); /* blinks LED */  
        T0IF = 0; /* clear timer 0 interrupt flag */  
    }  
}
```

- When timer interrupt is triggered, foreground program calls the interrupt timer handler routine.
- The `blink()` function will be called at each timer interrupt.
- The address of the `timer_handler` is stored in the interrupt vector table.
- The control returns to the foreground program after `timer_handler` executes.
- None of the given options

Questions...

- Which of the following statements is correct regarding I/O operations?
 - With busy wait, CPU can do useful work in parallel with the I/O
 - Memory-mapped I/O provides a separate address space for I/O devices
 - Busy-wait is asking an I/O device whether it is finished by reading its status register
 - I/O instructions use the CPU's normal memory read and write instructions to communicate with the devices
 - None of the given options

Questions...

- Which of the following option does not reduce the power consumption of embedded systems?
 - Increasing the clock frequency of the CPU
 - Decreasing the sampling rate of the analog-to-digital converter (ADC).
 - Internally disabling certain function units that are not required
 - Reducing the operating voltage level of the CPU
 - None of the given options

Questions...

- Which option below can be **incorrect** for embedded systems initialization?
 - Program execution is faster in RAM than executing code directly out of the flash memory
 - The reset vector jumps to the first instruction of the .text section of the boot image
 - Boot image copies initialized and uninitialized data sections (.data and .bss) of the flash memory into RAM
 - Constant data is part of the .const section which is read-only
 - None of the given options

Questions...

- Which of the following code lines are equivalent to the code below?

```
#define HWREG16(x) (*(volatile uint16_t*)(x))  
#define HWREG32(x) (*(volatile uint32_t*)(x))  
#define TA0CTL (HWREG16(0x40000000))  
TA0CTL = 0x0202;
```

- `volatile uint16_t* ta0_ctrl = (uint16_t*)0x40000000;
*ta0_ctrl = 0x0202;`
- `volatile uint16_t* ta0_ctrl = (uint16_t*)0x40000000;
ta0_ctrl = 0x0202;`
- `volatile uint32_t* ta0_ctrl = (uint32_t*)0x40000000;
ta0_ctrl = 0x0202;`
- `volatile uint16_t ta0_ctrl = (uint16_t*)0x40000000;
*ta0_ctrl = 0x0202;`
- None of the options

Questions...

- Which of the following statements is **incorrect** regarding interrupts in MSP432 platform?
 - Vector table is an array of addresses of instructions that cause an interrupt
 - Interrupt Service Routine is a function to be called in response to an interrupt request
 - Interrupt vector table is placed at address 0x0 in code memory
 - Interrupts are special events that request the CPU to perform a specific operation
 - None of the given options

Questions...

- What is the size of the following struct?

```
struct struct name {  
    int16_t var1;  
    int32_t var2;  
    int8_t var3;  
} __attribute__((packed));
```

- 8 bytes
- 6 bytes
- 7 bytes
- 12 bytes
- None of the given options

Questions...

- Assume that `buffer_full` is a variable that can be changed by an ISR or a device can directly modify it. What can be a problem for this program?
 - The variable `count` will never change
 - The while loop can be infinitely long
 - Function will return immediately
 - ISRs will never interrupt the program
 - None of the above.

```
int buffer_full=0;
int read_stream(void) {
    int count = 0;
    while (!buffer_full)
    {
        count++;
    }
    return count;
}
```

Questions

- Consider the following system of periodic processes executing on a single CPU. What is the CPU utilization of this system using RMS?

<u>Process</u>	<u>Execution time</u>	<u>Deadline</u>
P1	4	20
P2	1	5
P3	2	4
P4	1	10

- ☐ 100
- ☐ 95
- ☐ 85
- ☐ 90
- ☐ None of the given options

Questions

- Select the rank of the processes according to their priorities for a system that implements RMS (Rate Monotonic Scheduling). Rank from high-priority to low-priority.

<u>Process</u>	<u>Execution time</u>	<u>Deadline</u>
P1	4	20
P2	1	5
P3	2	4
P4	1	10

- P3, P2, P4, P1
- P1, P4, P2, P3
- P4, P2, P3, P1
- P1, P3, P2, P4
- None of the given options

Questions

- Which one below is a correct schedule for a system implementing Earliest Deadline First scheduling (EDF) in the first 8 timer ticks?

<u>Process</u>	<u>Execution time</u>	<u>Deadline</u>
P1	1	3
P2	1	4
P3	2	12
P4	2	8

- P1,P2,P4, P1,P2,P4,P3,P2
- P1,P2,P4, P1,P2,P4,P3(2)
- P1(4), P3(4)
- P4,P2, P3(3),P2,P3,P1
- None of the given options

Questions

- Which of the following does not reduce energy consumption:
 - Remove redundant operations
 - Help the compiler do a better optimization
 - Exploit hardware-specific features
 - Reduce floating point operations
 - None of the above

Questions

- Consider the following FreeRTOS pseudo-code where task T2 has greater priority than T1. Which of the following option is incorrect?
 - Priority inversion occurs since task T1 with low-priority prevents task T2 from running.
 - Task T1 executes after task T2 is blocked and takes the semaphore sem.
 - Task T2 executes first and blocks by calling xTaskDelay
 - Task T2 unblocks and but cannot get the semaphore sem since it is already taken by Task T1.
 - Task T2 will never get semaphore sem.

```
sem = xSemaphoreCreate(1,1); // semaphore with max
value 1 and initial value 1

void T1(void) {
    while (1) {
        xSemaphoreTake(sem);
        ... // do some stuff
        xTaskDelay(200ms); // block task for 200 ms
        xSemaphoreGive(sem);
    }
}

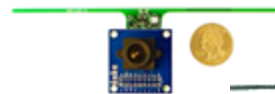
void T2(void) {
    while (1) {
        xTaskDelay(100ms); // block task for 100 ms
        xSemaphoreTake(sem);
        ... // do some stuff
        xSemaphoreGive(sem);
    }
}
```

Interested in Master's Degree?



Ambient Energy Sources

- If you are motivated in the following topics:
 - Software systems and programming support for low-power embedded computing
 - Operating systems, runtime libraries, compilers, programming languages...
 - Architectural (hardware) support for low-power embedded computing
 - FPGAs, new memory technologies
 - Batteryless and low-power wireless networking
 - Physical layer and MAC layer protocols...
 - Tiny machine learning on the edge.



Batteryless Devices

Low-Power Embedded Systems

- Spring Semester
 - Energy-efficient intelligent applications on low-power edge devices.
 - TinyML, batteryless computing, binary and spiking neural networks, and neuromorphic computing....